



Prompts

Pick reusable prompts and collect their parameter values.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-prompts



When this library is useful

Pick reusable prompts and collect their parameter values.

01

Choose a favorite

Browse
FavoritePromptItem
data supplied by the
host.

02

Fill placeholders

Collect values for
named prompt
parameters.

03

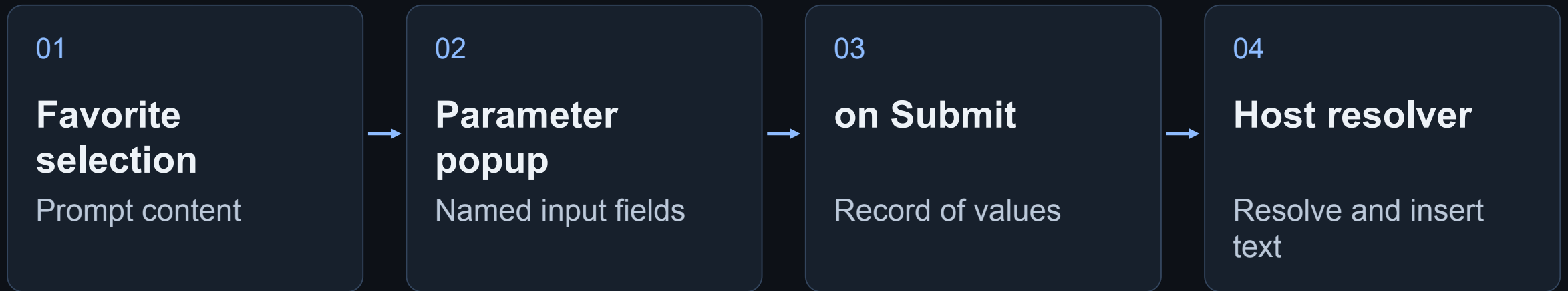
Populate a composer

Resolve values and
pass the text to host
input state.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Favorite prompt selection and parameter-entry UI.



The public contract to start with

PromptParametersPopup · selected surface; full API index is in the bundle.

Input or operation	Contract
content	string
parameters	PromptParameter[]
onSubmit	(values: Record<string, string>) => void
open	boolean



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
<PromptParametersPopup
  open={open} promptName="Demo summary"
  content="Summarize {{topic}}"
  parameters={[{ name: 'topic' }]}
  onClose={close} onCancel={close}
  onSubmit={values => {
    onText(resolvePromptParams(content, values));
  }}
/>
```

Full example: [examples/prompts.tsx](#) · uses [@epam/ai-dial-prompts](#)



Wire a realistic host workflow

Keep prompts focused on its public responsibility.

01

Collect parameters

Open the parameter popup with content and parameter names.

02

Resolve the template

Use the submitted value map with `resolvePromptParams`.

03

Apply the text

The host inserts the resolved text into its composer.



Available customization

01

Favorite callbacks

Selection, favorite changes and browse actions are caller-owned.

02

Default values

Shared helpers support tokens with a default value.

03

Explicit popup state

open, onClose and onCancel are controlled by the host.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	chat/components/PromptSelector/PromptParametersPopupOverlay.tsx
Test evidence	renders the title and one field per parameter
Test evidence	renders the "My Collection" header



Constraints that affect integration

01

Submission is values

The parameter popup returns a value map for the host to resolve.

02

Use shared helpers

Shared helpers extract and resolve prompt parameters.

03

Syntax matters

Double braces define parameters; ordinary braces remain literal.